

一、实验内容

1. 实验目标

本实验以微信小程序云开发为基础，实现一个名为“拾光 FRAME”的生活影像分享社区。项目没有直接照搬教程中的简单图片上传示例，而是在云存储、云数据库和云函数的基础上，补充了较完整的内容发布与社交交互流程。

主要目标如下：

1. 掌握微信小程序云开发环境的创建、初始化和调用方法。
2. 使用云存储保存用户上传的图片，使用云数据库保存作品和互动数据。
3. 使用云函数识别用户身份，并完成发布、删除、点赞、收藏和评论等操作。
4. 设计发现页、收藏页、个人主页、作品详情页和发布页，形成完整的页面跳转与数据流。
5. 在免费云开发环境的限制下，采用“数据库仅创建者读写、客户端通过云函数访问”的方式保护数据。
6. 改进教程中的基础界面，使小程序具有统一的视觉风格、清晰的信息层级和更好的移动端交互体验。

2. 项目概述

“拾光 FRAME”用于记录和分享生活中的照片。用户可以浏览不同类别的作品，搜索标题、描述或作者，发布最多9张图片，并对作品进行点赞、收藏和评论。收藏内容会显示在独立页面，个人主页会统计发布数量和收藏数量，同时展示用户自己的作品。

项目包含以下主要功能：

- 发现页双列作品流、关键词搜索和分类筛选；
- 多图发布、图片预览、删除、封面标记和草稿保存；
- 作品详情、图片轮播、点赞、收藏、评论、分享和保存图片；
- 收藏页与微信账号同步；
- 个人主页、作者主页及作品数量统计；
- 云端身份校验、事务更新和旧数据兼容；
- 本地演示数据与云端数据两种运行模式。

3. 开发环境与项目结构

本实验使用微信开发者工具完成，云开发环境名称为 `cloud1`，环境ID为 `cloud1-d9g4fxvqm538871d2`。小程序前端采用原生 WXML、WXSS 和 JavaScript，后端使用微信云函数和云数据库。



图1 微信云开发环境配置

项目的主要目录结构如下：

```
hope006/
├─ cloudfunctions/
│   └─ photoService/          # 作品、互动和用户身份相关云函数
├─ miniprogram/
│   ├── components/
│   │   └─ photo-card/        # 作品卡片组件
│   ├── images/nav/           # 底部导航图标
│   ├── pages/
│   │   ├── index/            # 发现页
│   │   ├── saved/            # 收藏页
│   │   ├── profile/          # 我的页面
│   │   ├── add/              # 发布页面
│   │   ├── detail/           # 作品详情页
│   │   └─ homepage/          # 作者主页
│   ├── utils/store.js         # 本地数据与云端请求封装
│   ├── app.js
│   ├── app.json
│   └─ config.js
└─ tests/regression.cjs        # 回归测试
```

4. 云开发环境初始化

在 `config.js` 中集中保存云环境ID，在 `app.js` 的启动阶段初始化云能力。集中配置可以避免多个页面重复填写环境ID，也便于在不同环境之间切换。

```
// config.js
module.exports = {
  envId: 'cloud1-d9g4fxvqm538871d2'
}
```

```
// app.js
const config = require('./config')

App({
  onLaunch() {
    if (wx.cloud) {
      wx.cloud.init({
        env: config.envId,
        traceUser: true
      })
    }
  }
})
```

由于免费环境中数据库权限选择有限，本项目将 `photos` 集合设置为“仅创建者可读写”。普通用户不直接读写集合，而是统一调用云函数。云函数可以从微信上下文中获得可信的 `OPENID`，因此不需要相信客户端自行传入的用户ID。

5. 数据结构设计

所有作品保存在 `photos` 集合中。为了减少集合数量，同时方便一次读取作品详情，点赞、收藏和评论数据也保存在作品文档内。

字段	类型	说明
<code>_id</code>	String	云数据库自动生成的作品ID
<code>ownerOpenId</code>	String	发布者的微信身份标识
<code>authorName</code>	String	作者昵称
<code>authorAvatar</code>	String	作者头像地址
<code>title</code>	String	作品标题
<code>description</code>	String	作品描述
<code>category</code>	String	自然、城市、旅行、日常或人像
<code>location</code>	String	拍摄地点
<code>images</code>	Array	云存储文件ID数组
<code>likeUsers</code>	Array	点赞用户的 OPENID 数组
<code>favoriteUsers</code>	Array	收藏用户的 OPENID 数组
<code>comments</code>	Array	评论对象数组
<code>createdAt</code>	Date	发布时间

评论对象包含评论ID、评论者 OPENID、昵称、头像、内容和时间。作品列表返回给前端时，云函数还会计算 `likeCount`、`favoriteCount`、`commentCount`、`liked` 和 `favorited` 等字段，页面无需自行推测云端状态。

6. 发现页设计

发现页是小程序的主要入口。页面顶部使用简洁的标题和说明，下方提供搜索框与类别筛选栏。作品区采用双列卡片布局，以较大的图片作为视觉中心，并显示类别、标题、作者头像、作者昵称、点赞按钮和点赞数量。

搜索支持标题、描述、地点和作者名称。输入内容后可以立即过滤作品，清除按钮固定在搜索框最右侧，避免图标随文字长度移动。类别包括“全部、自然、城市、旅行、日常、人像”，每个类别都准备了演示内容，使页面在首次启动时也有完整的展示效果。

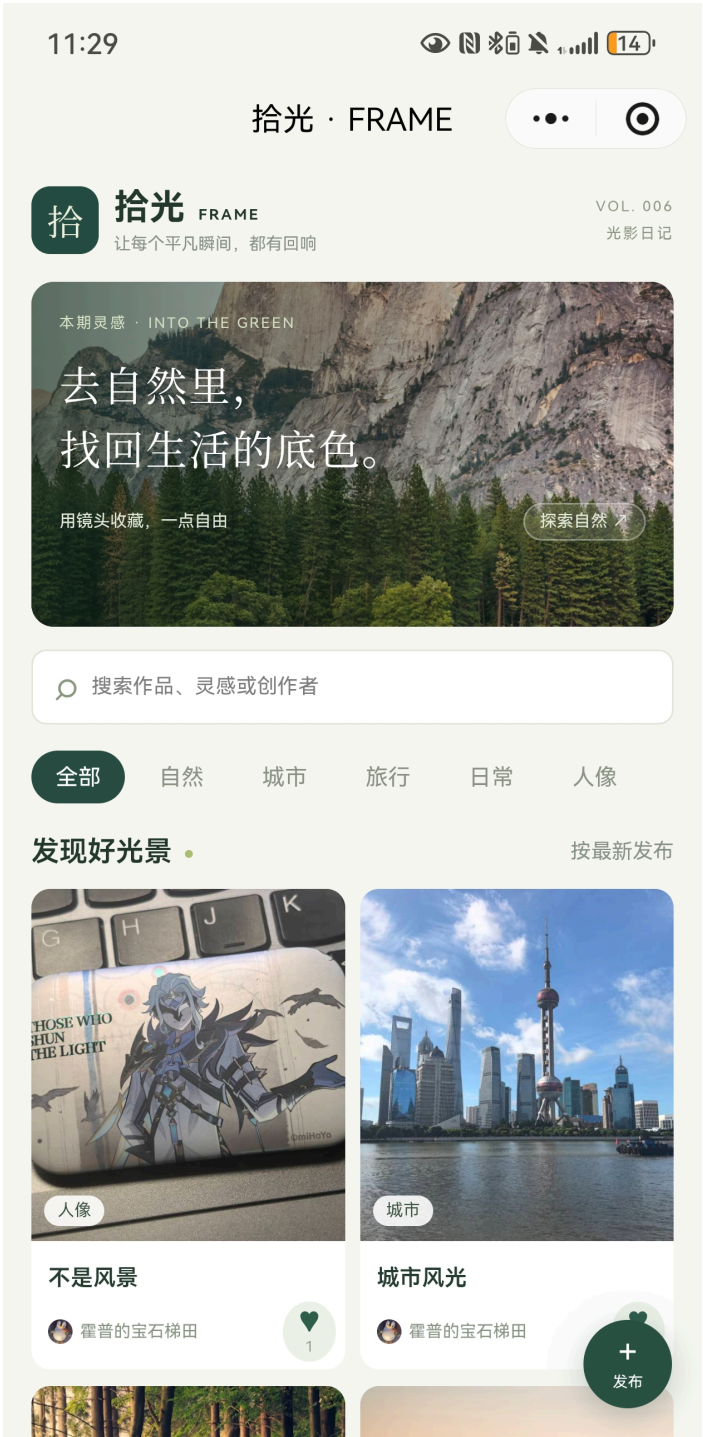


图2 发现页：搜索、分类、双列作品与点赞数量

作品卡片封装为 photo-card 组件，发现页、收藏页和个人主页可以复用相同的排版。点击图片或标题进入详情页，点击作者信息进入作者主页，点击右下角爱心则直接点赞。首页显示点赞数量，收藏和评论操作集中在详情页完成。

7. 多图片发布功能

发布页面允许用户一次选择最多9张图片。已选择的图片和“添加照片”入口放在同一个三列网格中，入口始终紧跟在最后一张图片之后，并保持相同的正方形尺寸。图片数量使用独立的横向文本布局显示为“1 / 9”，避免数字在窄屏设备上换行。



图3 发布作品页面与九宫格图片入口

选择图片后，用户可以预览或删除图片。第一张图自动作为封面，并在缩略图左下角显示“封面”标签。发布时先逐张上传到云存储，再将返回的文件ID和表单信息交给云函数写入数据库。

```
const fileIDs = []

for (let i = 0; i < tempFiles.length; i += 1) {
  const cloudPath = `photos/${Date.now()}-${i}.jpg`
  const result = await wx.cloud.uploadFile({
    cloudPath,
    filePath: tempFiles[i]
  })
  fileIDs.push(result.fileID)
}

await wx.cloud.callFunction({
  name: 'photoService',
  data: {
    action: 'publish',
    payload: {
      title,
      description,
      category,
      location,
      images: fileIDs
    }
  }
})
```

页面同时提供“存草稿”功能。用户临时退出后，标题、描述、类别、地点和可恢复的图片信息可以继续保留，减少重复输入。

8. 云函数与身份安全

所有云端业务集中在 `photoService` 云函数中，通过 `action` 参数分发操作。主要操作包括：

action	功能
<code>identity</code>	获取当前用户身份及个人统计
<code>list</code>	获取作品列表并进行搜索、分类和分页
<code>detail</code>	获取一篇作品及互动状态
<code>favoriteList</code>	获取当前用户收藏的作品
<code>publish</code>	发布新作品
<code>remove</code>	删除自己的作品及云存储文件
<code>toggleLike</code>	点赞或取消点赞
<code>toggleFavorite</code>	收藏或取消收藏

action	功能
<code>addComment</code>	添加评论
<code>removeComment</code>	删除自己的评论

云函数通过下面的方式获得当前调用者的身份：

```
const cloud = require('wx-server-sdk')
cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV })

exports.main = async (event) => {
  const { OPENID } = cloud.getWXContext()
  const { action, payload = {} } = event

  // 根据 action 执行对应业务，权限判断始终使用 OPENID
}
```

发布和删除时都会校验作品归属。即使客户端修改请求参数，也不能伪造其他用户身份或删除他人的内容。新增评论时限制内容长度和单篇评论总数，删除评论时检查评论者身份。

9. 点赞、收藏和评论同步

互动功能需要在不同页面和不同账号之间保持一致。为此，点赞和收藏不只保存在本机缓存中，而是写入云数据库。云函数使用数据库事务读取作品当前状态，然后增加或移除当前用户的 OPENID，并返回最新数量。

```
const result = await db.runTransaction(async transaction => {
  const snapshot = await transaction.collection('photos').doc(photoId).get()
  const users = Array.isArray(snapshot.data.likeUsers)
    ? snapshot.data.likeUsers
    : []

  const liked = users.includes(OPENID)
  const nextUsers = liked
    ? users.filter(id => id !== OPENID)
    : users.concat(OPENID)

  await transaction.collection('photos').doc(photoId).update({
    data: { likeUsers: nextUsers }
  })

  return { liked: !liked, likeCount: nextUsers.length }
})
```

使用事务可以降低多人同时操作时覆盖数据的风险。同一用户重复点击会在点赞与取消点赞之间切换，不会产生重复记录。收藏采用同样的处理方式。

详情页展示点赞、收藏和评论三个入口。评论发布成功后，云端立即返回新评论列表和评论数；评论区显示昵称、头像、内容与时间，用户还可以删除自己的评论。



图4 作品详情页及点赞、收藏、评论功能



图5 评论发布与云端同步结果

10. 收藏页与跨页面状态更新

收藏页只显示当前微信账号收藏的作品，并在页面顶部显示收藏总数。进入页面或从详情页返回时，页面会重新读取云端数据，因此取消收藏、他人新增点赞等变化都能更新到最新状态。

早期实现曾直接复用收藏时保存的旧作品对象，导致卡片上的点赞数停留在旧值。最终改为只根据 `favoriteUsers` 判断收藏关系，再从作品集合读取最新作品数据，解决了“收藏页显示的他人点赞没有同步”的问题。

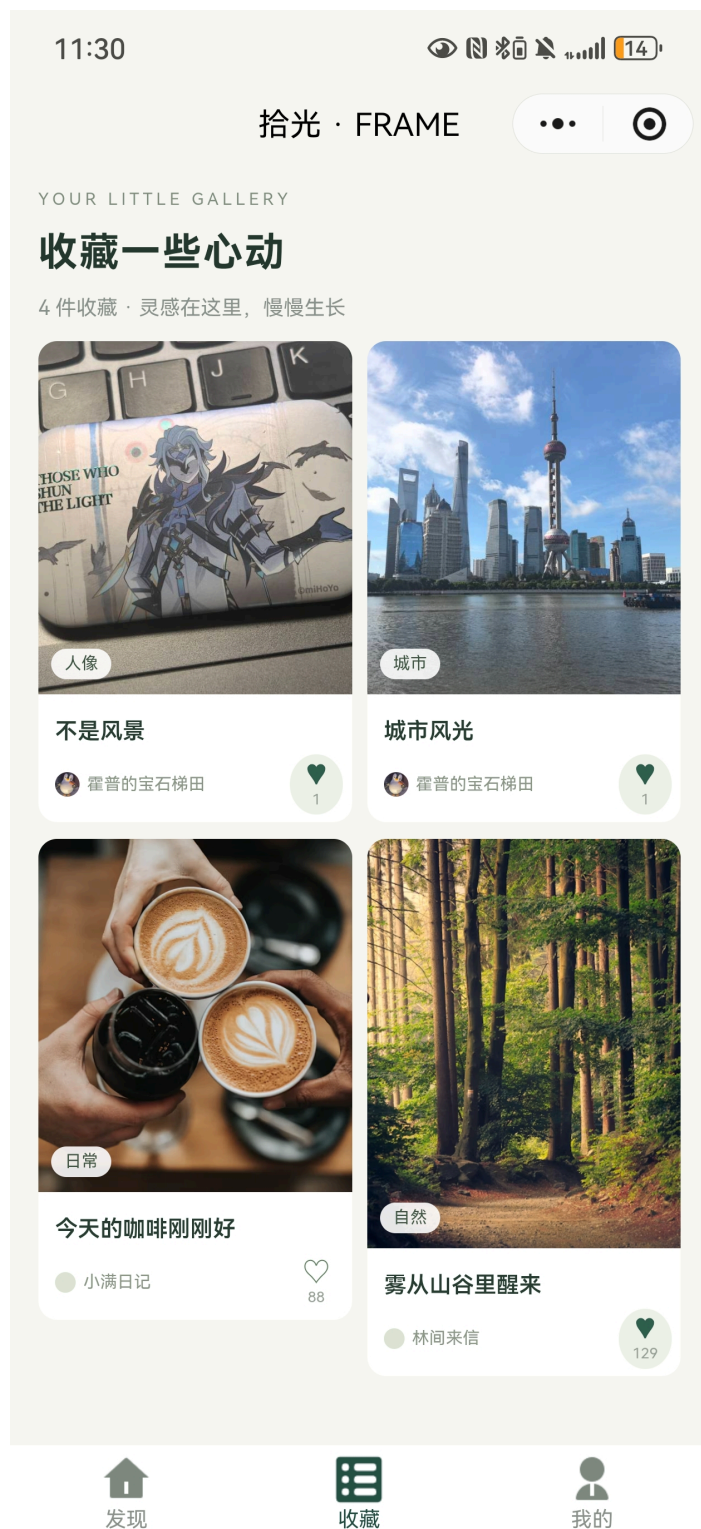


图6 收藏页与最新点赞数量

11. 个人主页与作者主页

“我的”页面顶部显示圆形头像、昵称和个人介绍。统计卡片只展示用户真正关心的内容：发布数量与收藏数量，不再显示“云端空间”等开发信息。自己的作品显示在“我的光影集”中，右侧提供尺寸适中的“发布作品”入口。

页面右下角还设置了圆形悬浮发布按钮。按钮的下边距和右边距保持一致，使其与屏幕边缘形成稳定留白，同时避开底部导航栏和安全区。



图7 个人主页、作品统计与发布入口

点击任意作品的作者信息可进入作者主页。作者主页展示该作者的头像、昵称、简介和公开作品，便于从单篇作品继续浏览同一作者的内容。

12. 作品详情与图片能力

作品详情页使用轮播组件展示多张图片，并标明当前页数。页面提供以下图片操作：

- 点击图片进入全屏预览；
- 将图片保存到手机相册；

- 通过微信分享作品；
- 作者删除自己的作品；
- 点赞、收藏和评论，并显示实时数量。

保存图片时先取得云文件的临时地址，再下载到本地并调用相册接口。如果用户尚未授予相册权限，页面会根据微信接口返回结果给出提示。

13. 视觉与交互优化

本项目使用低饱和米白和墨绿色作为主色，搭配圆角卡片、柔和阴影和较充足的留白，使图片成为页面重点。相比教程中的基础表单，项目进一步完成了以下细节：

1. 统一发现、收藏、个人主页和详情页的字体、颜色与圆角。
2. 头像固定为圆形并放在个人信息左侧，昵称和介绍横向排列。
3. 发布按钮缩小为圆润的悬浮按钮，并根据安全区调整位置。
4. 发布页采用真正的三列网格，图片和添加入口连续排列。
5. 底部“发现、收藏、我的”三个标签加入普通与选中状态图标。
6. 搜索框清除按钮固定在最右侧，并扩大可点击区域。
7. 对昵称进行解码和兜底处理，避免出现 %E9%... 形式的编码文本。
8. 对旧作品缺少互动字段的情况提供默认值，升级后仍能正常浏览。

14. 测试与验证

完成开发后，从页面编译、数据流、云端权限和交互状态四个方面进行了验证。

检查项目	验证内容	结果
WXML编译	9个页面及组件模板	通过
WXSS编译	10个页面及组件样式文件	通过
页面事件	点击、输入、滚动和跳转事件绑定	通过
本地数据模式	搜索、分类、收藏、多图发布和删除	通过
云端身份	OPENID 获取及客户端身份伪造防护	通过
文件校验	图片数量、文件ID与输入字段校验	通过
点赞收藏	幂等切换、数量更新和跨页面同步	通过
评论功能	添加、数量更新及禁止跨用户删除	通过

此外，使用模拟云开发 SDK 执行了自动回归测试，覆盖搜索关键词转义、多图上传进度、作品删除、事务更新、旧数据兼容等情况。最终编译结果为 WXML 9个文件、WXSS 10个文件全部通过，回归测试全部通过。

二、问题总结与体会

问题及解决

1. 云数据库权限与免费环境限制

问题： 免费云开发环境中可选权限较少。如果直接允许所有用户读写，作品和互动数据容易被客户端任意修改；如果设置为仅创建者读写，普通用户又不能直接读取他人作品。

解决方法： 将数据库设置为“仅创建者可读写”，所有业务操作统一放入云函数。云函数使用微信提供的可信 `OPENID` 判断用户身份，并在服务器端完成查询和更新。这样既满足实验环境限制，也比直接开放数据库更安全。

2. 收藏页中的点赞数没有及时同步

问题： 收藏列表最初保存的是收藏瞬间的作品快照。当其他用户点赞后，发现页能看到新数量，收藏页却仍显示旧值。

解决方法： 收藏关系只保存用户ID，展示收藏列表时重新读取最新作品文档，并在页面显示时刷新数据。这样收藏页、发现页和详情页都以同一份云端作品数据为准。

3. 点赞、收藏和评论的并发更新

问题： 如果先在客户端读取数组，再直接覆盖数据库，多名用户同时操作时可能丢失其中一次更新。

解决方法： 在云函数中使用事务完成“读取当前数据—计算新数组—写回文档”的完整过程，并对数组进行去重。评论还增加了长度、数量和删除权限检查。

4. 发布页九宫格排版异常

问题： 初始版本使用普通弹性布局和原生 `button` 组件实现添加图片入口。原生按钮自带最小尺寸与内部样式，导致添加入口和缩略图之间出现大间距，数量“1 / 9”也会被挤成多行。

解决方法： 将图片列表改为固定三列的 CSS Grid，把添加入口改为普通 `view`，并让图片与入口共用相同的 `218rpx` x `218rpx` 网格尺寸。数量部分使用横向 `flex` 和禁止换行的文本节点，最终可以稳定显示在一行。

5. 昵称出现百分号编码

问题： 部分演示数据或缓存中的中文昵称经过 URL 编码后直接显示，页面出现 `%E9%9C...` 一类文本。

解决方法： 在数据标准化阶段识别并安全解码昵称；遇到空值或解码失败时使用默认昵称。处理逻辑放在数据层后，所有页面和卡片组件都能获得一致结果。

6. 底部导航图标不显示

问题： 自定义生成的图标在部分微信开发者工具版本中没有正常渲染，并且工具缓存会继续显示旧资源。

解决方法： 改用兼容性更好的 PNG 图标，为普通和选中状态分别着色，同时更换资源文件名以刷新缓存。三个标签最终都具有清晰的图标与选中状态。

收获与体会

本次实验我学会了微信小程序云开发。不只是把图片上传到云端，我还认真设计用户身份、数据库权限、数据结构和页面刷新时机。我的项目中拓展了点赞、收藏和评论功能，这些要实现多账号同步时涉及事务、幂等操作、旧数据兼容和跨页面状态更新等。

在界面开发中，我也体会到移动端排版需要反复在真实尺寸下检查。头像方向、按钮边距、九宫格间距和一行文字是否换行，都会直接影响使用感受。将重复的作品卡片抽成组件、把云端操作集中到一个数据层之后，修改样式和排查问题都更加容易。

最终完成的“拾光 FRAME”已经不再是单一的上传示例，而是包含发现、发布、详情、互动、收藏和个人主页的完整小程序。通过这次实验，我掌握了云存储、云数据库、云函数和原生小程序组件的配合方式，也进一步理解了在有限的实验环境中如何兼顾功能、数据安全和用户体验。